Jaypee Institute of Information Technology University, Noida

Department of Computer Science Engineering

Project Synopsis

NetWatch: Linux Network Activity and Security Monitoring System



Submitted by:

Akshara Yadav - 992501230047

Shambhavi Singh - 992501230049

Labhansh Vashisht - 992501230050

Mohd Nauman Ali - 992501230060

Course: Unix Programming Lab
Course Code: 24B15CS214
Program: Btech CSE (Cyber Security)
2nd Year (3rd Semester)

Table of Contents

1. Introduction / Background 3
2. Problem Statement 3
3. Objectives 3
4. Proposed Methodology / Approach 4
Step 1: Collect Network Information 4
Step 2: Parse and Process Data 4
Step 3: Process-to-Network Mapping 4
Step 4: Real-Time Monitoring 5
Step 5: Event Detection and Alerts 5
Step 6: Logging 6
Step 7: Command-Line Interface 6
High-Level Flow 6
5. Unix/Linux Commands and Concepts to be Used 7
5.1 Linux Commands / Utilities 7
5.2 Unix/Linux Concepts 7
6. Functional Modules 8
Module 1: Network Connection Monitor 9
Module 2: Listening Port Monitor 9
Module 3: Process-to-Port Mapper 9
Module 4: Real-Time Event Monitor 9
Module 5: Alert and Rule Engine 10
Module 6: Logging and History 10
Module 7: Command-Line Interface 10
7. Expected Outcome 10
8. Software / Hardware Requirements 11
8.1 Software Requirements 11
8.2 Hardware Requirements 11
9. Future Scope 11
10. References 12

1. Introduction / Background

Modern Linux systems continuously communicate over networks through multiple TCP and UDP connections, listening ports, and background services. Although standard utilities such as ss, netstat, lsof, and /proc provide information about network activity, their output is generally command-oriented and requires users to manually interpret different sources of information.

NetWatch is a Unix/Linux-based network monitoring and security analysis tool designed to provide a centralized view of network activity on a Linux system. The system will monitor active network connections, listening ports, connection states, and the processes associated with network activity. It will also maintain logs and generate alerts when potentially suspicious or unexpected network events are detected.

The project focuses on applying Unix/Linux programming concepts such as processes, file descriptors, signals, /proc filesystem information, pipes, command execution, file handling, and shell utilities to develop a practical system-monitoring application.

2. Problem Statement

Linux provides several powerful commands and system interfaces for inspecting network activity, but the information is distributed across different commands and files. A user may need to execute multiple commands and manually correlate a port or network connection with the process responsible for it.

This makes continuous monitoring and identification of unexpected network activity inconvenient, especially for users who need a simple overview of what is happening on their system.

The proposed system aims to address this problem by developing a single Linux-based monitoring tool that:

- Displays active network connections and their states.
- Identifies listening ports on the system.
- Associates network activity with the corresponding process or PID wherever possible.
- Detects newly appearing listening ports or unusual connection events.
- Maintains logs of observed network activity.
- Provides alerts for configurable suspicious events.

3. Objectives

The main objectives of NetWatch are:

1. To develop a Linux-based tool for monitoring network connections and listening ports.
2. To display TCP and UDP network activity in an organized manner.
3. To identify the local and remote endpoints involved in active connections.
4. To determine the process/PID associated with network activity where system information permits.

5. To monitor changes in listening ports and active connections.
6. To generate alerts for configurable or potentially suspicious network events.
7. To maintain a log of important network events for later analysis.
8. To demonstrate the practical use of Unix/Linux commands, system interfaces, processes, file descriptors, and system programming concepts.
9. To provide a simple command-line interface through which users can view and analyze network activity.

4. Proposed Methodology / Approach

The project will be implemented as a modular Linux command-line application. The development is divided into seven steps, described below.

Step 1: Collect Network Information

NetWatch will collect information about active connections, listening ports, protocols, IP addresses, ports, and connection states using Linux system interfaces and utilities such as `/proc`, `ss`, and related system information.

Step 2: Parse and Process Data

The collected information will be parsed and converted into a structured internal representation containing the following fields:

Field	Description
Protocol	TCP or UDP
Local IP address	Address of the local endpoint
Local port	Port number on the local system
Remote IP address	Address of the remote endpoint
Remote port	Port number on the remote system
Connection state	For example LISTEN, ESTABLISHED, TIME_WAIT
Process ID	PID of the owning process
Process name	Name of the owning process

Step 3: Process-to-Network Mapping

Where available, NetWatch will correlate network information with process information. This will allow the system to present information such as:

```
Port 3000 → PID 2841 → node
Port 22   → PID 912  → sshd
```

Each socket in /proc/net/tcp and /proc/net/udp carries an inode number. By scanning the file descriptors of each process under /proc/[pid]/fd and matching socket inodes, the owning PID and process name can be determined, as illustrated in Figure 1.

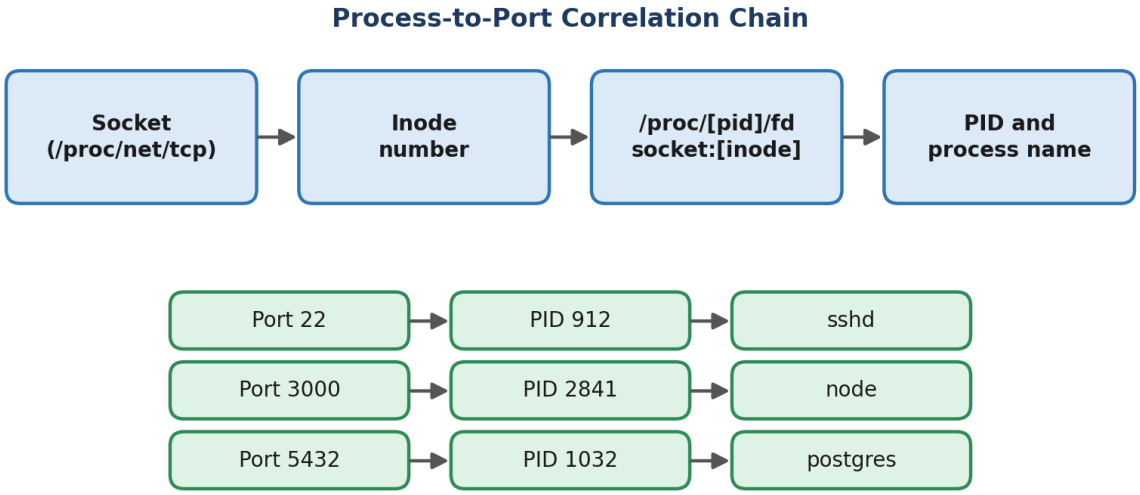


Figure 1: Process-to-network correlation chain

Step 4: Real-Time Monitoring

The application will periodically inspect the system and compare the current state with previously observed information. Changes such as a newly opened listening port or a newly detected connection can then be identified. The monitoring cycle is shown in Figure 2.

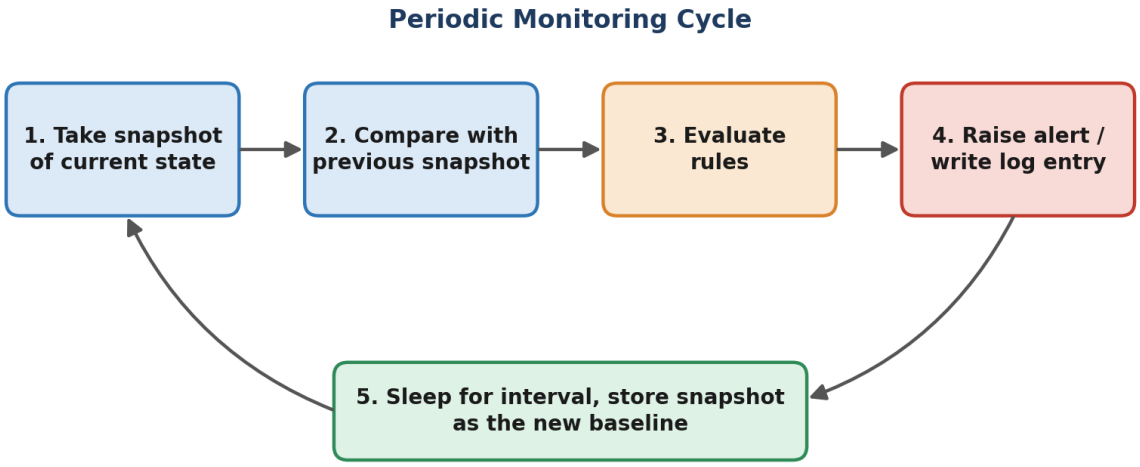


Figure 2: Periodic monitoring cycle

Step 5: Event Detection and Alerts

Configurable rules will be used to identify events of interest, such as:

- A new listening port.

- A new connection.
- A large number of repeated connection events.
- A process appearing with network activity.
- An unexpected port or service, according to user-defined rules.

Step 6: Logging

Important events will be stored in a log file with information such as timestamp, event type, protocol, address, port, and associated process where available.

Step 7: Command-Line Interface

The user will be able to start monitoring, display current connections, inspect listening ports, view alerts, and access historical logs through a simple terminal interface.

High-Level Flow

Figure 3 summarizes the overall data flow of the system, from raw Linux system information to the user-facing dashboard.

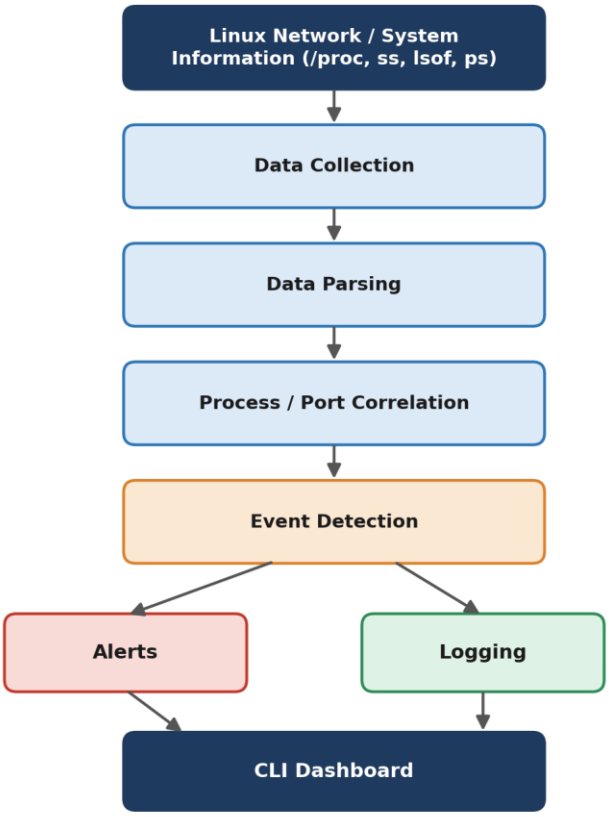


Figure 3: High-level data flow of NetWatch

5. Unix/Linux Commands and Concepts to be Used

5.1 Linux Commands / Utilities

The project may use the following commands and utilities as supporting system interfaces:

Command	Purpose
`ss`	Display socket and network connection information.
`netstat`	Reference utility for viewing network statistics where available.
`lsof`	Identify files and sockets used by processes.
`ps`	Display process information.
`grep`	Filter text-based system information.
`awk`	Process and extract structured fields from command output.
`cat`	Read system information files.
`kill`	Demonstrate interaction with processes where required.
`date`	Obtain timestamps for logging.
`tail`	Inspect monitoring logs.

5.2 Unix/Linux Concepts

The following system programming and networking concepts will be demonstrated:

System Programming	Networking and Environment
Processes and Process IDs (PIDs)	TCP and UDP concepts
Process creation using fork()	IP addresses and port numbers
Program execution using exec() family	Socket states
Synchronization using wait() / waitpid()	/proc pseudo-filesystem
Signals and signal handling	Command-line arguments
File descriptors	Shell scripting
File I/O: open(), read(), write(), close()	Background process execution
Pipes and inter-process communication	Permissions and system-level access
Periodic monitoring and event logging	—

Figure 4 illustrates how process creation, program execution, and pipes can be combined so that NetWatch obtains the output of utilities such as ss, lsof, and ps.

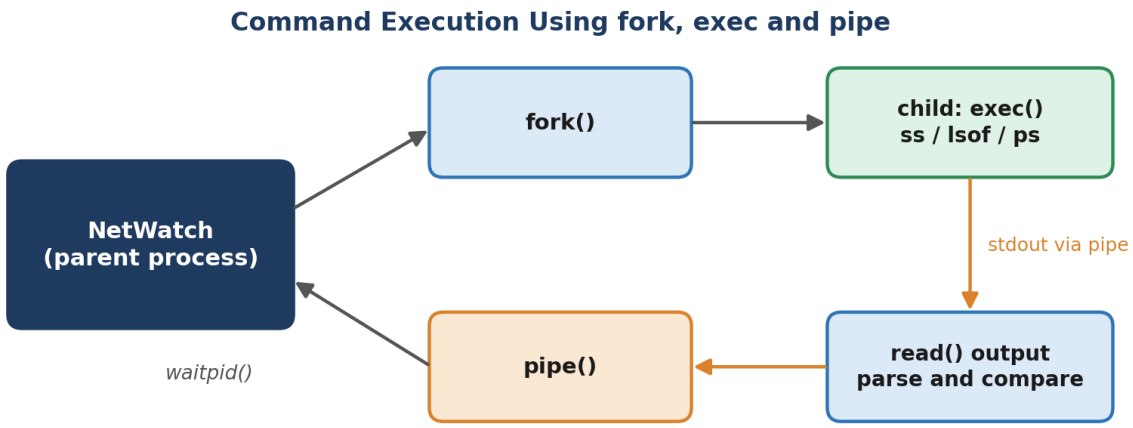
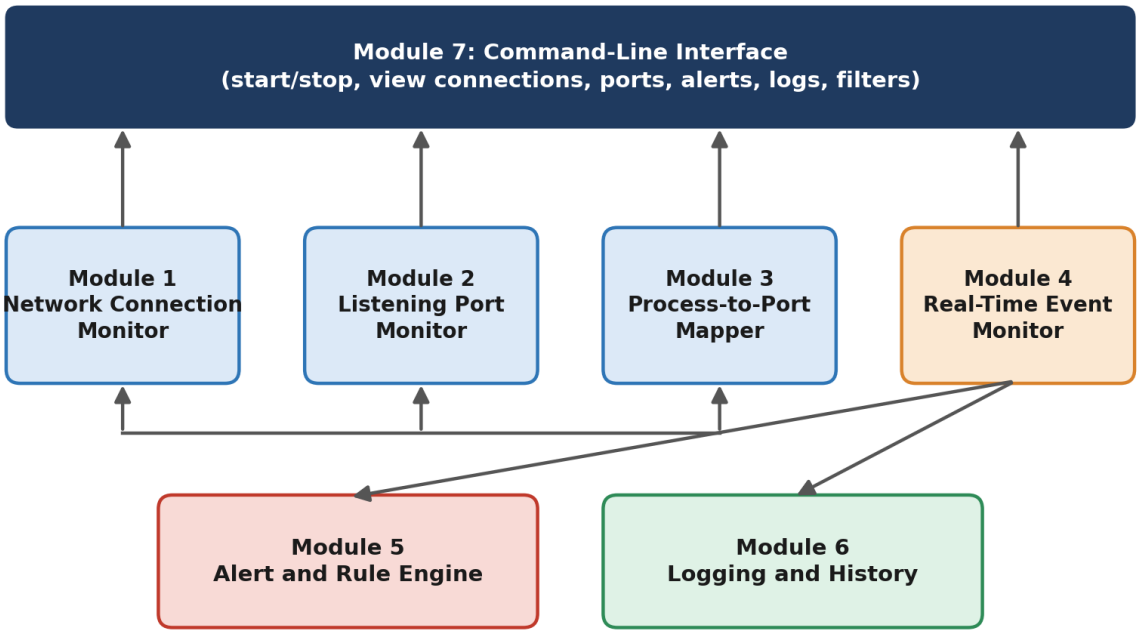


Figure 4: Command execution and data capture using fork(), exec() and pipe()

The exact system calls and commands used will depend on the final implementation and Linux environment.

6. Functional Modules

NetWatch is organized into seven cooperating modules, shown in Figure 5. Data-gathering modules feed the real-time event monitor, which passes events to the alert engine and the logger; all functionality is exposed through the command-line interface.



Data sources: /proc/net/, /proc/[pid]/fd, ss, lsof, ps*

Figure 5: Functional module architecture

Module 1: Network Connection Monitor

Displays currently active TCP and UDP connections, including protocol, local endpoint, remote endpoint, and connection state.

Module 2: Listening Port Monitor

Displays ports on which services are currently listening and identifies changes in the listening-port list.

Module 3: Process-to-Port Mapper

Attempts to associate network sockets with their corresponding process ID and process name. Example output:

```
PORT     PID      PROCESS
22       912      sshd
3000     2841     node
5432     1032     postgres
```

Module 4: Real-Time Event Monitor

Continuously checks for changes in network activity and identifies newly detected events.

Module 5: Alert and Rule Engine

Applies configurable rules to monitored events and generates warnings for events that require attention. Example output:

```
[ALERT] New listening port detected: 4444
[ALERT] New network process detected: python3 (PID 2841)
```

Module 6: Logging and History

Stores important monitoring events with timestamps and relevant network/process information.

Module 7: Command-Line Interface

Provides commands/options for:

- Starting and stopping monitoring.
- Viewing active connections.
- Viewing listening ports.
- Viewing alerts.
- Viewing logs.
- Filtering network information.

7. Expected Outcome

At the completion of the project, NetWatch is expected to provide a functional Linux command-line monitoring system capable of:

- Displaying active network connections.
- Displaying listening ports.
- Showing TCP/UDP connection information.
- Associating network activity with processes where possible.
- Detecting changes in network activity.
- Generating configurable alerts.
- Recording important network events in log files.
- Providing users with a consolidated view of network activity without requiring them to manually execute multiple commands.

The project will also demonstrate practical knowledge of Unix/Linux system programming, process management, file handling, /proc filesystem usage, networking concepts, and command-line tools.

8. Software / Hardware Requirements

8.1 Software Requirements

Component	Requirement
Operating System	Linux (Ubuntu recommended)
Programming Language	C / C++
Compiler	GCC / G++
Shell	Bash
Build Tool	Make / CMake (optional)
Text Editor / IDE	VS Code, Vim, Nano, or equivalent
Linux Utilities	ss, ps, grep, awk, lsof, and related tools
Version Control	Git (recommended)

8.2 Hardware Requirements

Component	Requirement
Processor	Dual-core or better
RAM	Minimum 4 GB
Storage	Minimum 5 GB free space
Network Interface	Ethernet or Wi-Fi
System	Any standard laptop/desktop capable of running a modern Linux distribution

9. Future Scope

The project can be extended with the following features:

1. **Graphical User Interface:** Develop a GUI dashboard showing connections, ports, processes, and alerts visually.
2. **Advanced Rule Configuration:** Allow users to create custom rules based on IP addresses, ports, processes, protocols, and connection frequency.
3. **Network Traffic Statistics:** Add bandwidth and packet-related statistics where supported by the operating system.
4. **Historical Analytics:** Provide summaries and charts of network activity over time.
5. **Process Control:** Allow authorized users to terminate or restrict selected processes associated with suspicious activity.

6. **IP Reputation Integration:** Integrate external threat-intelligence sources to flag potentially malicious IP addresses.
7. **Remote Monitoring:** Extend the system to monitor multiple Linux machines from a centralized interface.
8. **Notification System:** Add email or desktop notifications for critical events.
9. **Configuration Profiles:** Provide predefined monitoring profiles for servers, development systems, and personal Linux machines.
10. **Security Hardening:** Improve privilege handling, access control, input validation, and secure logging.

10. References

[1] Linux manual pages (man pages) for ss, ps, lsof, grep, awk, kill, fork, exec, waitpid, open, read, write, and proc.

[2] Linux Programmer's Manual / man7.org documentation.

[3] Michael Kerrisk, The Linux Programming Interface, No Starch Press.

[4] W. Richard Stevens, Stephen A. Rago, Advanced Programming in the UNIX Environment, Addison-Wesley.

[5] Abraham Silberschatz, Peter B. Galvin, Greg Gagne, Operating System Concepts, Wiley.

[6] Linux kernel documentation for networking and the /proc filesystem.